

ONEDRAFT

CAD-AI — Aria Powered

Identity, Access Control & Security System

Version 0.1

Status: Implementation Specification

Security Model: Multi-Tenant RBAC + Capability Authorization

Authentication: Bearer Token / OIDC-Compatible

Authorization: Organization / Project / Resource Scope

Identifiers: UUID

Primary Security Boundary: Organization → Project → Resource

AI Security Boundary: Aria → Application Capabilities → Domain Services

1. SECURITY DESIGN PRINCIPLE

Security is a foundational subsystem of OneDraft rather than an external feature added after implementation.

Every operation entering the platform must establish:

Who is making the request

What organization they belong to

What project they are accessing

What resource they are attempting to access

What operation they are attempting to perform

Whether the operation is authorized

Whether the operation is permitted in the current project state

The security system therefore operates before domain mutation, document generation, geometry execution, compliance execution, artifact access, and Aria command execution.

The security architecture becomes:

IDENTITY



AUTHENTICATION



ORGANIZATION MEMBERSHIP



PROJECT AUTHORIZATION
↓
RESOURCE AUTHORIZATION
↓
CAPABILITY AUTHORIZATION
↓
DOMAIN OPERATION
↓
AUDIT EVENT

2. SECURITY DOMAINS

The OneDraft security subsystem shall initially contain:

identity
authorization
security
audit
system

The existing:

identity.users
identity.organizations
identity.organization_members

remain the authoritative identity records.

The security subsystem extends those records without redefining the underlying identity architecture.

3. SECURITY PRINCIPLES

OneDraft shall implement:

Least privilege

Explicit authorization

Project isolation

Organization isolation

Deny by default

Capability-based execution

Immutable security events

Credential separation

Service identity separation

Short-lived access tokens

Auditable privileged operations

No direct database access by external clients

4. AUTHENTICATION BOUNDARY

External clients authenticate against the OneDraft authentication boundary.

The architecture is:

```
CLIENT
↓
AUTHENTICATION PROVIDER
↓
ACCESS TOKEN
↓
API GATEWAY
↓
AUTHORIZATION
↓
DOMAIN SERVICES
```

The domain services do not independently implement separate user authentication systems.

Authentication establishes identity.

Authorization establishes permission.

These concerns must remain separate.

5. USER IDENTITY

The authoritative user record remains:

`identity.users`

The security layer associates authenticated identities with:

```
user_id
organization_id
membership
roles
scopes
session
authentication_events
```

No service may create an independent user identity for normal application access.

6. SERVICE IDENTITIES

Internal services shall have independent identities.

Examples:

```
api-service
runtime-orchestrator
geometry-worker
compliance-worker
validation-worker
documentation-worker
artifact-service
event-bus
aria-orchestrator
notification-service
```

A service identity is distinct from a human user.

Service actions therefore record:

```
actor_type = SERVICE
actor_id = service_identity_id
```

Human actions record:

```
actor_type = USER
actor_id = user_id
```

7. SERVICE IDENTITY TABLE

identity.service_identities

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(255) NOT NULL
service_type VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
description TEXT
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(name)
```

Indexes:

idx_service_identities_status
idx_service_identities_type

8. SERVICE CREDENTIALS

Credentials shall never be stored directly in application source code.

Conceptually:

identity.service_credentials

Fields:

```
id UUID PRIMARY KEY
service_identity_id UUID NOT NULL
credential_type VARCHAR(64) NOT NULL
credential_reference TEXT NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
expires_at TIMESTAMPTZ
revoked_at TIMESTAMPTZ
```

The actual secret material should be maintained through a dedicated secret-management system.

The database stores references and lifecycle metadata rather than plaintext credentials.

9. ROLES

OneDraft shall initially support organization and project roles.

Initial organization roles:

```
OWNER
ADMIN
MEMBER
BILLING_ADMIN
AUDITOR
```

Initial project roles:

```
PROJECT_OWNER
PROJECT_ADMIN
DESIGNER
DRAFTER
REVIEWER
COMPLIANCE_REVIEWER
DOCUMENT_MANAGER
VIEWER
```

Roles represent human-readable authorization groupings.

They do not replace capability checks.

10. ROLE TABLE

authorization.roles

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(128) NOT NULL
scope_type VARCHAR(32) NOT NULL
description TEXT
system_defined BOOLEAN NOT NULL DEFAULT FALSE
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

Unique:

```
UNIQUE(name, scope_type)
```

11. ROLE PERMISSIONS

authorization.role_permissions

Fields:

```
role_id UUID NOT NULL
permission_id UUID NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

Primary key:

```
(role_id, permission_id)
```

This creates the relationship:

```
ROLE
↓
PERMISSIONS
```

12. PERMISSIONS

authorization.permissions

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(128) NOT NULL
resource_type VARCHAR(64) NOT NULL
action VARCHAR(64) NOT NULL
description TEXT
system_defined BOOLEAN NOT NULL DEFAULT FALSE
created_at TIMESTAMPTZ NOT NULL
```

Examples:

```
project.read
project.write
```

```
model.read
model.write
model.modify
```

```
geometry.read
geometry.execute
```

```
compliance.read
compliance.execute
```

```
validation.read
validation.execute
```

```
drawing.read
drawing.generate
```

```
document.read
document.generate
```

```
redline.read
redline.create
redline.resolve
```

```
aria.request
aria.execute
aria.approve
```

```
acceptance.create
```

```
audit.read
```

13. ORGANIZATION ROLE ASSIGNMENTS

identity.organization_roles

Fields:

```
organization_id UUID NOT NULL
user_id UUID NOT NULL
role_id UUID NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
```

updated_at TIMESTAMPTZ NOT NULL

Primary key:

(organization_id, user_id, role_id)

14. PROJECT ROLE ASSIGNMENTS

authorization.project_roles

Fields:

project_id UUID NOT NULL
user_id UUID NOT NULL
role_id UUID NOT NULL
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL

Primary key:

(project_id, user_id, role_id)

This permits project-level access without granting access to every project belonging to the organization.

15. RESOURCE AUTHORIZATION

Authorization must occur at multiple levels.

ORGANIZATION

↓

PROJECT

↓

RESOURCE

↓

OPERATION

For example:

A user may have permission to:

project:read

without having permission to:

aria:execute

Likewise, a user may have:

aria:execute

within Project A while having no authorization against Project B.

16. PROJECT TENANCY ISOLATION

Every project operation must resolve:

organization_id
project_id
actor_id

before accessing project data.

The system must prevent cross-organization resource access even where a valid UUID is supplied.

A valid object identifier does not constitute authorization.

The authorization boundary must verify ownership through the project hierarchy.

17. RESOURCE OWNERSHIP

Resource ownership is derived wherever possible.

Example:

drawing
↓
project
↓
organization

Therefore:

drawing.project_id

determines the authorization boundary.

The system must not trust a client-supplied:

organization_id

as proof of ownership.

18. CAPABILITY AUTHORIZATION

High-impact operations shall use explicit capabilities.

Examples:

```
CREATE_PROJECT
MODIFY_MODEL
EXECUTE_ARIA_COMMAND
EXECUTE_GEOMETRY
EXECUTE_COMPLIANCE
EXECUTE_VALIDATION
GENERATE_DRAWING
GENERATE_DOCUMENT
RESOLVE_REDLINE
ACCEPT_DOCUMENT_PACKAGE
```

A capability is granted only after the authorization service confirms the actor's permissions.

19. ARIA SECURITY BOUNDARY

Aria shall never receive unrestricted application authority.

The architecture remains:

```
USER
↓
ARIA_REQUEST
↓
ARIA_ORCHESTRATOR
↓
CAPABILITY_AUTHORIZATION
↓
DOMAIN_COMMAND
↓
DOMAIN_SERVICE
↓
TRANSACTION
```

Aria may propose or request operations.

The application decides whether those operations are permitted.

20. AI COMMAND AUTHORIZATION

Every Aria command must contain:

```
project_id
actor_id
command_id
```

operation
target_objects
parameters
expected_revision

The command boundary verifies:

authentication
authorization
project membership
object ownership
revision currency
constraint validity
operation capability

before execution.

21. PRIVILEGED AI OPERATIONS

Certain operations require elevated authorization.

Examples:

DELETE_PROJECT
CHANGE_PROJECT_OWNER
MODIFY_SECURITY_CONFIGURATION
CHANGE_ORGANIZATION_MEMBERSHIP
MODIFY_BILLING
ALTER_FINALIZED_DOCUMENT
OVERRIDE_VALIDATION
OVERRIDE_COMPLIANCE

Aria must never bypass these controls.

Where an operation requires explicit human approval, the workflow becomes:

ARIA PROPOSAL
↓
SECURITY CHECK
↓
HUMAN APPROVAL
↓
COMMAND EXECUTION
↓
VALIDATION
↓
AUDIT

22. ACCESS TOKENS

The API shall accept bearer access tokens.

Conceptually:

Authorization: Bearer <access_token>

Tokens should contain or resolve to:

subject
organization
scopes
issued_at
expires_at
issuer
audience
token_id

Tokens must have finite lifetimes.

Long-lived credentials shall not be used as normal API access tokens.

23. TOKEN VALIDATION

The API gateway validates:

signature
issuer
audience
expiration
not-before
token status

The authorization service subsequently evaluates:

organization membership
project membership
roles
permissions
resource scope

Authentication success alone does not authorize the requested operation.

24. SESSION MANAGEMENT

Where interactive sessions are supported, the system shall maintain session metadata.

security.sessions

Fields:

```
id UUID PRIMARY KEY
user_id UUID NOT NULL
organization_id UUID
token_reference VARCHAR(255)
created_at TIMESTAMPTZ NOT NULL
last_seen_at TIMESTAMPTZ
expires_at TIMESTAMPTZ NOT NULL
revoked_at TIMESTAMPTZ
status VARCHAR(32) NOT NULL
```

Sessions may be revoked independently.

25. AUTHENTICATION EVENTS

Authentication activity shall be recorded.

Examples:

```
LOGIN_SUCCESS
LOGIN_FAILURE
TOKEN_ISSUED
TOKEN_REVOKED
SESSION_CREATED
SESSION_REVOKED
PASSWORD_CHANGED
MFA_ENABLED
MFA_DISABLED
```

These events belong to the security audit stream.

26. SECURITY EVENTS

`audit.security_events`

Fields:

```
id UUID PRIMARY KEY
actor_type VARCHAR(32) NOT NULL
actor_id UUID
organization_id UUID
project_id UUID
event_type VARCHAR(128) NOT NULL
resource_type VARCHAR(64)
resource_id UUID
result VARCHAR(32) NOT NULL
reason TEXT
metadata JSONB
timestamp TIMESTAMPTZ NOT NULL
```

The table is append-only.

27. AUTHORIZATION DECISION LOG

For security-sensitive operations the system should retain the authorization decision.

`security.authorization_decisions`

Fields:

```
id UUID PRIMARY KEY
request_id UUID NOT NULL
actor_type VARCHAR(32) NOT NULL
actor_id UUID
organization_id UUID
project_id UUID
resource_type VARCHAR(64)
resource_id UUID
requested_action VARCHAR(128) NOT NULL
decision VARCHAR(16) NOT NULL
policy_reference VARCHAR(255)
reason TEXT
timestamp TIMESTAMPTZ NOT NULL
```

Possible decisions:

```
ALLOW
DENY
REQUIRES_APPROVAL
```

28. REQUEST CORRELATION

Every authenticated request receives:

`request_id`

The request identifier propagates through:

```
API
↓
Runtime Orchestrator
↓
Job
↓
Worker
↓
Event
↓
Artifact
↓
```

Audit

This permits reconstruction of a complete execution chain.

29. SECURITY CONTEXT

Every internal request should carry a security context conceptually containing:

```
request_id
actor_type
actor_id
organization_id
project_id
roles
permissions
scopes
source_service
```

Services must not reconstruct authorization from arbitrary client parameters.

The trusted security context is established at the service boundary.

30. INTERNAL SERVICE AUTHORIZATION

Internal services must authenticate to one another.

The architecture must not assume that:

```
internal network = trusted
```

The system therefore uses service identity verification for internal requests.

Example:

```
API
↓
RUNTIME ORCHESTRATOR
```

The orchestrator verifies the API service identity.

Likewise:

```
RUNTIME ORCHESTRATOR
↓
GEOMETRY WORKER
```

the worker verifies the calling service.

31. WORKER SECURITY

Workers must execute only authorized jobs.

A worker receives:

```
job_id
service_identity
project_id
model_version_id
authorization_context
```

The worker verifies that the job belongs to an authorized service workflow before execution.

A worker must not accept arbitrary user instructions directly.

32. EVENT BUS SECURITY

Events published to the event bus must contain trusted metadata.

Conceptually:

```
event_id
event_type
producer
project_id
organization_id
request_id
timestamp
payload
```

Consumers must validate the event producer before acting on privileged events.

33. ARTIFACT ACCESS CONTROL

Generated artifacts inherit project authorization.

For example:

```
PROJECT
↓
DOCUMENT PACKAGE
↓
PDF
```

A user authorized to access the project may retrieve the package only if the user's current permissions permit document access.

Artifact storage must not become an authorization bypass.

34. SIGNED ARTIFACT ACCESS

Where object storage is used, direct object access should be mediated through controlled access mechanisms.

The API may issue short-lived signed retrieval credentials after authorization.

The architecture becomes:

```
CLIENT
↓
API AUTHORIZATION
↓
SIGNED ACCESS
↓
OBJECT STORAGE
```

The storage system itself does not determine project membership.

35. DATABASE SECURITY

External clients never connect directly to PostgreSQL.

The architecture remains:

```
CLIENT
↓
API
↓
DOMAIN SERVICES
↓
DATABASE
```

Database credentials are service-specific.

Services receive only the database privileges required for their function.

36. DATABASE ROLE SEPARATION

Initial database roles may include:

```
onedraft_api
onedraft_worker
onedraft_migration
onedraft_readonly
```

The migration role possesses elevated schema privileges.

Runtime services should not use migration credentials.

37. ROW-LEVEL SECURITY

PostgreSQL Row-Level Security should be considered for high-value tenant boundaries.

Where implemented:

organization_id
project_id

become policy inputs.

Application-level authorization remains mandatory.

Database-level controls provide defense in depth rather than replacing application authorization.

38. SECRET MANAGEMENT

Secrets shall be externalized from:

source code
database rows
container images
OpenAPI documents
logs
event payloads

Examples include:

database credentials
AI provider credentials
OIDC secrets
object storage credentials
event bus credentials
signing keys
encryption keys

Secrets are referenced through the deployment environment or dedicated secret-management infrastructure.

39. ENCRYPTION

Sensitive communication shall use encrypted transport.

Conceptually:

TLS

is required for external API communication.

Sensitive persistent data should use encryption at rest through the underlying infrastructure.

Encryption keys must be managed independently from application data.

40. PERSONAL DATA MINIMIZATION

OneDraft should retain only identity information necessary for platform operation.

Examples:

email
display_name
organization membership
authorization metadata
audit identity

Application logs should not unnecessarily reproduce sensitive user information.

41. LOGGING SECURITY

Logs must not contain:

access tokens
passwords
private keys
secret credentials
raw authentication secrets

Request identifiers should be logged instead.

Example:

request_id = UUID
actor_id = UUID
operation = GENERATE_DOCUMENT
status = COMPLETED

42. SECURITY AUDIT TRAIL

Security-sensitive operations generate immutable audit events.

Examples:

USER_CREATED
USER_DISABLED
MEMBER_ADDED
MEMBER_REMOVED
ROLE_ASSIGNED
ROLE_REVOKED
TOKEN_REVOKED
PROJECT_ACCESS_GRANTED
PROJECT_ACCESS_REVOKED
ARIA_COMMAND_AUTHORIZED
ARIA_COMMAND_DENIED
DOCUMENT_ACCESS_GRANTED
DOCUMENT_ACCESS_DENIED

43. ADMINISTRATIVE OPERATIONS

Administrative actions must be explicitly identified.

Examples:

organization administration
membership administration
role administration
security policy administration
billing administration
project ownership transfer

Administrative actions receive enhanced audit records.

44. BREAK-GLASS ACCESS

A future controlled emergency-access mechanism may exist for system operators.

Any such access must require:

explicit activation
reason
operator identity
time limit
scope
complete audit trail

Break-glass access must never become a permanent authorization path.

45. FINALIZED PROJECT SECURITY

A finalized document package requires stronger controls.

Once finalized:

```
PROJECT
↓
FINAL REVISION
↓
VALIDATION
↓
DOCUMENT PACKAGE
↓
ACCEPTANCE
```

Subsequent modifications create a new revision rather than silently modifying the finalized state.

Authorization therefore works together with revision control.

46. SECURITY AND REVISION INTEGRATION

A command must verify both:

AUTHORIZATION

and:

EXPECTED_REVISION

Example:

```
authorized user
+
expected_revision = 17
+
current_revision = 17
```

permits execution.

If:

```
expected_revision = 17
current_revision = 18
```

the command is rejected as stale regardless of authorization.

47. SECURITY AND COMPLIANCE INTEGRATION

Authorization does not override regulatory validation.

The command path remains:

```
AUTHORIZATION
↓
MODEL COMMAND
↓
CONSTRAINT CHECK
↓
COMPLIANCE / VALIDATION
↓
COMMIT
```

A user authorized to make a design change does not automatically authorize the system to accept an invalid design state.

48. SECURITY AND REDLINES

Redline creation and redline resolution are separately authorized operations.

For example:

```
redline:create
```

does not necessarily grant:

```
redline:resolve
```

Likewise:

```
redline:resolve
```

does not permit bypassing model validation.

49. SECURITY AND DOCUMENT GENERATION

Document generation requires:

```
project access
+
document generation permission
```

- +
valid revision
- +
required validation state

The document engine must reject unauthorized or invalid generation requests.

50. SECURITY AND CUSTOMER ACCEPTANCE

Acceptance is a controlled operation.

The acceptance workflow verifies:

- customer identity
- project authorization
- revision identity
- model version
- document package
- validation state

The acceptance record then becomes immutable except through an explicit superseding workflow.

51. API SECURITY ENDPOINTS

The initial API security surface shall include:

- GET /api/v1/me
- GET /api/v1/me/organizations
- GET /api/v1/me/projects
- GET /api/v1/projects/{project_id}/members
- POST /api/v1/projects/{project_id}/members
- PATCH /api/v1/projects/{project_id}/members/{user_id}
- DELETE /api/v1/projects/{project_id}/members/{user_id}
- GET /api/v1/projects/{project_id}/roles
- GET /api/v1/projects/{project_id}/permissions
- GET /api/v1/projects/{project_id}/security/events

Administrative endpoints require elevated permissions.

52. AUTHORIZATION MIDDLEWARE

The API shall implement centralized authorization middleware.

Conceptually:

```
request
↓
authenticate()
↓
load_security_context()
↓
resolve_project()
↓
authorize()
↓
domain_handler()
```

Domain handlers should not rely on each individual route implementing authentication independently.

53. CAPABILITY CHECK INTERFACE

The application should expose a common capability interface.

Conceptually:

```
authorize(
    actor,
    action,
    resource,
    project
)
```

The result becomes:

```
ALLOW
DENY
REQUIRES_APPROVAL
```

This prevents authorization logic from becoming duplicated throughout the application.

54. POLICY ENGINE

Authorization policies should eventually be represented independently from route implementations.

Conceptually:

```
POLICY
↓
ROLE
↓
PERMISSION
↓
CAPABILITY
↓
RESOURCE
```


This allows security policy to evolve without rewriting domain services.

55. SECURITY POLICY VERSIONING

Security policies may eventually be versioned.

Each authorization decision may therefore reference:

`policy_version`

This creates an auditable explanation of which security policy governed a decision.

56. AUTHORIZATION FAILURE

Unauthorized requests return a standard error.

Example:

```
{
  "error": {
    "code": "AUTHORIZATION_DENIED",
    "message": "The requested operation is not authorized for this project.",
    "resolution": "Contact a project administrator if additional access is
required."
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

The response must not disclose protected resource information.

57. AUTHENTICATION FAILURE

Authentication failures use a separate error category.

Example:

```
AUTHENTICATION_REQUIRED
INVALID_TOKEN
TOKEN_EXPIRED
TOKEN_REVOKED
```

Authentication and authorization failures must remain distinguishable internally even where external responses are intentionally generic.

58. RATE LIMITING

The API gateway should support rate limiting at multiple levels:

- IP
- user
- organization
- project
- service
- endpoint

Higher-risk endpoints receive stricter controls.

Examples:

- authentication
- ARIA execution
- document generation
- large exports
- administrative operations

59. ARIA RATE LIMITING

Aria execution shall have independent resource controls.

Limits may be based on:

- organization
- project
- user
- command type
- compute cost
- concurrent jobs

This prevents accidental or malicious command storms from overwhelming the geometry, validation, or documentation workers.

60. JOB AUTHORIZATION

A job inherits authorization context from its originating request.

The job record should contain or reference:

- request_id
- actor_id
- organization_id

project_id
authorization_context

Workers must not re-authorize against stale client input.

They verify the trusted job authorization context and current project state.

61. EVENT AUTHORIZATION

Not every event should be consumable by every service.

Event permissions should distinguish:

publish
consume
administrate
replay

Replay privileges require elevated authorization because replaying an event can cause domain operations to execute again.

62. EVENT REPLAY SECURITY

Event replay must require:

event_id
replay_reason
authorized_actor
target_environment
approval where required

Replay operations are independently audited.

63. ARTIFACT SECURITY

Artifacts shall have:

artifact_id
project_id
revision_id
model_version_id
artifact_type
storage_reference
content_hash
access_policy

The artifact service verifies project authorization before issuing access.

64. DOWNLOAD AUTHORIZATION

A document download request becomes:

```
REQUEST
↓
AUTHENTICATE
↓
AUTHORIZE PROJECT
↓
AUTHORIZE DOCUMENT
↓
VERIFY ARTIFACT
↓
ISSUE ACCESS
```

A storage URL must never be exposed permanently where the artifact is intended to remain access-controlled.

65. CONTENT INTEGRITY

Generated artifacts should retain:

`content_hash`

The hash becomes part of document provenance.

For final packages:

```
PROJECT
↓
REVISION
↓
MODEL VERSION
↓
CODE MANIFEST
↓
VALIDATION
↓
DOCUMENT PACKAGE
↓
CONTENT HASH
```

Security and provenance therefore remain connected.

66. SECURITY TESTING

The initial security test suite shall verify:

unauthenticated access rejected
invalid token rejected
expired token rejected
revoked token rejected
cross-organization access rejected
cross-project access rejected
insufficient role rejected
insufficient scope rejected
unauthorized Aria command rejected
stale revision rejected
unauthorized artifact access rejected
unauthorized document generation rejected
unauthorized acceptance rejected
worker impersonation rejected
event replay without authorization rejected

67. TENANT ISOLATION TEST

A mandatory integration test shall attempt:

Organization A

↓

Project A

to access:

Organization B

↓

Project B

using valid authentication credentials belonging to Organization A.

The request must fail.

No project object, document, geometry, event, or artifact from Project B may be disclosed.

68. ARIA SECURITY TEST

The test shall submit an Aria command requesting an operation for which the authenticated actor lacks authorization.

Expected result:

AUTHORIZATION_DENIED

No:

model mutation
revision
geometry job
drawing job
document

may be created.

The denial itself is audited.

69. PRIVILEGED COMMAND TEST

The test shall submit a privileged command without the required approval.

Expected result:

REQUIRES_APPROVAL

The system must not execute the command.

70. SECURITY REGRESSION TEST

Every future subsystem must preserve:

authentication
authorization
tenant isolation
project isolation
resource ownership
revision integrity
artifact protection
auditability

Security therefore becomes a cross-cutting regression requirement for all future OneDraft services.

71. IMPLEMENTATION MIGRATION ORDER

The initial security migration sequence should be:

```
026_service_identities
027_roles
028_permissions
029_role_permissions
030_organization_roles
031_project_roles
032_sessions
033_authorization_decisions
034_security_events
035_service_credentials
036_security_indexes
037_security_seed_data
```

Each migration is immutable once applied.

72. SECURITY SEED DATA

Development environments should contain:

```
OWNER
ADMIN
MEMBER
BILLING_ADMIN
AUDITOR

PROJECT_OWNER
PROJECT_ADMIN
DESIGNER
DRAFTER
REVIEWER
COMPLIANCE_REVIEWER
DOCUMENT_MANAGER
VIEWER
```

Associated permissions shall be seeded deterministically.

Development credentials must never be reused in production.

73. OPENAPI SECURITY DEFINITION

The OpenAPI contract shall define the bearer authentication scheme.

Conceptually:

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
```

Protected endpoints declare the required security mechanism.

74. API SECURITY HEADERS

The API should support:

Authorization
Idempotency-Key
X-Request-ID

Where appropriate, the server generates the request identifier if the client does not provide one.

The final request identifier is returned through response metadata.

75. SECURITY RESPONSE METADATA

Every security-sensitive response should contain:

request_id
timestamp

Where appropriate, administrative responses may additionally expose:

authorization_decision_id

76. SECURITY OBSERVABILITY

Security events shall integrate with the existing event bus.

Conceptually:

SECURITY EVENT
↓

EVENT BUS
↓
AUDIT
↓
OBSERVABILITY
↓
ALERTING

This allows security monitoring without coupling the authorization engine directly to every monitoring implementation.

77. SECURITY ALERT CONDITIONS

The platform should eventually detect patterns such as:

repeated authentication failures
repeated authorization failures
cross-project access attempts
cross-organization access attempts
unusual Aria command volume
unusual document generation volume
repeated privileged-operation denials
unexpected service identity activity
event replay anomalies

Detection remains separate from authorization.

78. SECURITY STATE MACHINE

Security-sensitive identities follow controlled state transitions.

Example:

ACTIVE
↓
SUSPENDED
↓
REVOKED

A revoked identity cannot return to active status without an explicit reactivation workflow.

Likewise:

ACTIVE PROJECT MEMBER

↓

ACCESS REVOKED

immediately prevents subsequent project operations.

Existing jobs must separately determine whether authorization remains valid for completion.

79. JOB REVOCATION

If a user's project access is revoked while a job is executing, the system must determine whether the job:

- continues
- pauses
- cancel s

according to job policy.

High-impact operations should revalidate authorization before committing final state.

This prevents authorization changes from being bypassed through queued execution.

80. SECURITY AND TRANSACTION COMMIT

For high-impact commands the final transaction boundary becomes:

BEGIN

- verify identity
- verify authorization
- verify project
- verify revision
- verify constraints
- apply mutation
- record revision
- record security decision
- record audit event

COMMIT

A failed authorization check produces:

ROLLBACK

No unauthorized state transition may remain.

81. SECURITY AND DISTRIBUTED EXECUTION

Because OneDraft uses asynchronous workers, security context must survive distributed execution.

The trusted execution chain becomes:

```
USER
↓
API REQUEST
↓
AUTHORIZED COMMAND
↓
JOB
↓
WORKER
↓
DOMAIN OPERATION
↓
VALIDATION
↓
FINALIZATION
↓
AUDIT
```

Each stage retains the originating:

```
request_id
project_id
actor_id
command_id
job_id
```

82. SECURITY PROVENANCE CHAIN

A complete operation should eventually be reconstructable as:

```
ACTOR
↓
AUTHENTICATION
↓
AUTHORIZATION DECISION
↓
COMMAND
↓
JOB
↓
MODEL REVISION
↓
GEOMETRY
↓
VALIDATION
```

↓
DOCUMENT
↓
ARTIFACT
↓
ACCEPTANCE

This establishes security provenance across the entire computational project lifecycle.

83. MVP SECURITY SUCCESS CRITERIA

The security implementation is successful when it can:

Authenticate a user

Authenticate an internal service

Create organization membership

Assign roles

Evaluate permissions

Authorize project access

Reject unauthorized project access

Authorize resource operations

Protect artifacts

Protect generated documents

Authorize Aria commands

Reject unauthorized Aria commands

Protect asynchronous jobs

Protect event consumption

Record security decisions

Record security events

Maintain tenant isolation

Maintain revision integrity

Support privileged operations

Audit administrative operations

84. FIRST SECURITY INTEGRATION TEST

The first complete security integration test should execute:

1. Create Organization A
2. Create Organization B
3. Create User A
4. Create User B
5. Add User A to Organization A
6. Add User B to Organization B
7. Create Project A
8. Create Project B
9. Grant User A project access to Project A
10. Authenticate User A
11. Read Project A
12. Attempt to read Project B
13. Verify authorization denial
14. Submit authorized Aria command to Project A
15. Verify command authorization
16. Submit unauthorized privileged command
17. Verify rejection
18. Generate document for Project A
19. Retrieve authorized artifact
20. Attempt unauthorized artifact access
21. Verify denial
22. Revoke User A project membership
23. Attempt Project A access
24. Verify denial
25. Retrieve security event history
26. Verify complete authorization audit trail

This becomes the first major OneDraft security regression test.

85. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now contains:

Product definition

System architecture

Technology architecture

Domain model

Persistence model

API contract

Geometry engine

Parametric model engine

Compliance engine

Validation engine

Documentation engine

Revision system

Runtime orchestrator

Job and worker execution

Artifact storage

Integration and event bus

Identity system

Authorization system

Security boundary

Security audit

AI command security boundary

Tenant isolation

The security layer therefore becomes the control plane governing access to the computational project system.

86. NEXT IMPLEMENTATION ARTIFACTS

The next engineering package shall consist of four machine-oriented artifacts:

Artifact A

`026_security_schema.sql`

The PostgreSQL migration establishing roles, permissions, service identities, sessions, authorization decisions, and security events.

Artifact B

`security_policy.py`

The centralized authorization and capability evaluation layer.

Artifact C

`security_context.py`

The authenticated actor, organization, project, scope, permission, and request context model.

Artifact D

`authorization_middleware.py`

The API and internal-service authorization boundary.

Together these artifacts establish the executable security control plane for OneDraft.

87. FINAL ARCHITECTURAL DECISION

OneDraft shall not treat security as:

LOGIN
+
PASSWORD
+
API TOKEN

Security shall instead operate as:

IDENTITY
+
AUTHENTICATION
+
TENANCY
+
ROLE
+

PERMISSION
+
CAPABILITY
+
RESOURCE AUTHORIZATION
+
REVISION AUTHORIZATION
+
SERVICE IDENTITY
+
AUDIT

The result is a security architecture capable of protecting the computational project model while preserving the distributed execution architecture already established.

88. SPECIFICATION STATUS

ONEDRAFT IDENTITY, ACCESS CONTROL & SECURITY SYSTEM v0.1

STATUS: ESTABLISHED

The next implementation stage is the creation of the executable security foundation:

`026_security_schema.sql`

`security_policy.py`

`security_context.py`

`authorization_middleware.py`

These artifacts constitute the first executable OneDraft security control plane.